Cache Memory Mapping

Cache Mapping:

The process /technique of bringing data of main memory blocks into the cache block is known as cache mapping.

The mapping techniques can be classified as :

- Direct Mapping(covered in the previous lecture)
- Fully Associative Mapping
- Set-Associative Mapping

Associative Mapping:

- Here the mapping of the main memory block can be done with any of the cache block.

- The memory address has only 2 fields here: word & tag.

- This technique is called as fully associative cache mapping.

Example: Fully Associative Mapping

If we have a fully associative mapped cache of 8 KB size with block size = 128 bytes and say, the size of main memory is = 64 KB.

⇒ Number of bits for the physical address = 16 bits (as memory size = 64 KB = $2^6 \times 2^{10} = 2^{16}$)

⇒ Number of bits in block offset = 7 bits (as block size = 128 bytes = 2^7)

⇒ No of tag bits = Number of bits for the physical address – Number of bits in block offset = 16-7 = 9 bits

⇒ No of cache Blocks = Cache size/block size = 8 KB / 128 Bytes = 8×1024 Bytes/128 Bytes = 2^6 blocks.

Another example

- Cache memory size= 64KB

  Block size= 32B

  Number of bits given for main memory addressing= 32

  Find number of bits required for tag and word-offset?

- Solution: -

  Block size=32B

  Block offset= $\log_2 2^5 = 5$

  Tag= 32-5=27

Set – Associative Mapping

- It is the combination of advantages of both direct & associative mapping.
- This is referred to as L-way set-associative mapping.

Draw back in direct mapping:-

- Compulsory miss occurs.
- conflict miss occurs.
- Cache memory could not be used effectively.

Draw back in fully-associative mapping:-

- Compulsory miss occurs.
- Capacity miss occurs.
- To over come the loopholes present in both mapping Set-associative mapping technique is used.

- To find the set number :-

set number= main memory block number % Number of sets in cache.

- Physical address is divided into three parts.
  - ✓ Block-offset
  - ✓ Set-offset
  - ✓ Tag

Example:2-way Set-Associative Mapping

Example: If we have a fully associative mapped cache of 8 KB size with block size = 128 bytes and say, the size of main memory is = 64 KB, and we have *"2-way"* set-associative mapping (Assume each word has 8 bits). Then :

Number of bits for the physical address = 16 bits (as memory size = 64 KB = 2^6 * 2^{10} = 2^{16})

No of cache Blocks = Cache size/block size = 8 KB / 128 Bytes = 8×1024 Bytes/128 Bytes = 2^6 cache blocks.

No of Main Memory Blocks = MM size/block size = 64 KB / 128 Bytes = 64×1024 Bytes/128 Bytes = 2^9 MM blocks.

No of sets of size 2 = No of Cache Blocks/ L = $2^6/2$ = 2^5 cache sets.(L = 2 as it is 2-way set associative mapping)

Example:Set-Associative Memory Mapping

- Cache memory size=64KB
  Main memory size=4GB
  Block size=32B , 4-way associative
  Find tag, set-offset and word-offset?

- Solution:- Number of blocks in cache = size of cache memory / Block size
  $=64KB / 32B = 2KB = 2^1 \times 2^{10} = 2^{11}$
  Number of sets in cache = no of blocks in cache / associativity
  $= 2KB/4 = 2^{11} / 2^2 = 2^9$
  Set offset = $\log_2 2^9 = 9$

Continued..

Block size =32B

Number of bits required for word offset= $\log_2 2^5 = 5$

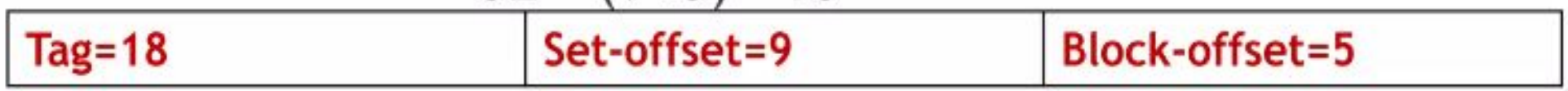
Size of main memory =4GB= $2^2 \times 2^{30} = 2^{32}$

Number of bits required for memory addressing

$= \log_2 2^{32} = 32$

Tag = 32-(set-offset + word-offset)

= 32 - (9+5) =18



Tag-directory size computation

- Tag-directory size = Tag x number of blocks

  $=18 \times 2^{11}$

  $= 36 \times 2^{10}$

  = 36 KB

- Tag-directory size= (tag + number of extra bits given) x (number of blocks)
- [if in question it is given that number of dirty_bits=1 and number of modified bit =2]
- Tag-directory size = $(18 + 1 + 2) \times 2^{11}$

  $= 21 \times 2^{11}$

  $= 42 \times 2^{10}$

  = 42KB

Differences and Comparative Analysis:

Direct-mapping	Associative Mapping	Set-Associative Mapping
Needs only one comparison because of using direct formula to get the effective cache address.	Needs comparison with all tag bits, i.e., the cache control logic must examine every block's tag for a match at the same time in order to determine that a block is in the cache/not.	Needs comparisons equal to number of blocks per set as the set can contain more than 1 blocks.
Main Memory Address is divided into 3 fields : TAG, **BLOCK** & WORD. The BLOCK & WORD together make an index. The least significant WORD bits identify a unique word within a block of main memory, the BLOCK bits specify one of the blocks and the Tag bits are the most significant bits.	Main Memory Address is divided into 1 fields : TAG & WORD.	Main Memory Address is divided into 3 fields : TAG, **SET** & WORD.

Differences and Comparative Analysis:

There is one possible location in the cache organization for each block from main memory because we have a fixed formula.	The mapping of the main memory block can be done with any of the cache block.	The mapping of the main memory block can be done with a particular cache block of any direct-mapped cache.
If the processor need to access same memory location from 2 different main memory pages frequently, cache hit ratio decreases.	If the processor need to access same memory location from 2 different main memory pages frequently, cache hit ratio has no effect.	In case of frequently accessing two different pages of the main memory if reduced, the cache hit ratio reduces.
Search time is less here because there is one possible location in the cache organization for each block from main memory.	Search time is more as the cache control logic examines every block's tag for a match.	Search time increases with number of blocks per set.

Differences and Comparative Analysis:

The index is given by the number of blocks in cache.	The index is zero for associative mapping.	The index is given by the number of sets in cache.
It has least number of tag bits.	It has the greatest number of tag bits.	It has less tags bits than associative mapping and more tag bits than direct mapping.
Advantages- • Simplest type of mapping • Fast as only tag field matching is required while searching for a word. • It is comparatively less expensive than associative mapping.	**Advantages-** • It is fast. • Easy to implement	**Advantages-** • It gives better performance than the direct and associative mapping techniques.
Disadvantages- • It gives low performance because of the replacement for data-tag value.	**Disadvantages-** • Expensive because it needs to store address along with the data.	**Disadvantages-** • It is most expensive as with the increase in set size cost also increases.

Questions?